Quarter-Car Suspension Modeling and Simulation

In this project, you will use MATLAB and Simscape Multibody to build and tune a quarter-car suspension model using an automated road test suite. You'll first develop the model using Simscape Multibody to represent the mechanical structure and motion of a vehicle corner (body corner + wheel assembly connected through suspension and tire compliance). Then, you'll create a test suite in MATLAB of road profiles that automatically computes objective metrics for comfort and road holding. Finally, you'll tune the suspension parameters to meet constraints and improve performance across all road cases.

Use this Live Script as a template to get started on your project solution. You may include helper functions at the bottom of this document or as separate files. For a tutorial on how to use Live Scripts, check out this video.

Suggested Tasks

1. Build a baseline quarter-car multibody model in Simscape
2. Create a road test suite of 3-5 road profiles
3. Log signals and define objective metrics
4. Build an automated test runner
5. Tune suspension parameters
6. Validate robustness with a simple variation

Table of Contents

Break Down the Problem

In the space below, define the problem statement:

List the requirements, constraints, and success criteria for the project:

What is your proposed solution?

List the steps you will need to take to achieve your proposed solution:

What challenges do you face in the process of achieving the solution?

Task 1: Build a baseline quarter-car multibody model

In Simscape Multibody,

Build two rigid bodies:

- Sprung mass (vehicle body corner)
- Unsprung mass (wheel assembly)

Connect them with:

- Suspension spring + damper (parameterized)
- Tire vertical compliance element (parameterized or fixed)

Provide a road displacement input (e.g., a vertically moving “road plate” or equivalent road excitation subsystem)

Tip: Keep geometry simple (vertical DOF-focused) so the model is stable and fast to simulate.

```
% Open Simulink to build your vehicle model in Simscape Multibody
% Optionally, provide code below to open your .slx model and run the
% simulation

open_system('your_vehicle_model.slx'); % open model
simOut = sim('your_vehicle_model.slx', 'ReturnWorkspaceOutputs', 'on') %
run simulation and return results
```

Task 2: Create a road test suite (3-5 cases)

Implement a set of road displacement profiles such as:

- Speed bump (smooth hump)
- Pothole (smooth dip)
- Rough road (band-limited noise)
- (optional) Washboards (sinusoidal corrugation)
- (optional) Two bumps (repeatability / transient recovery)

Below are a few options for building the road suite (choose one approach or mix them):

MATLAB-generated signals (recommended):

- Create each road profile as a `timeseries` or `[t, z]` array in `roadSuite.m`
- Examples: speed bump (half-sine hump over a fixed duration), pothole (negative half-sine dip), rough road (filtered noise using `randn` + a simple filter)
- Feed into the model using From Workspace (or Signal Editor) blocks.

Simulink blocks (no MATLAB scripting required):

- speed bump / pothole: Signal Builder or Signal Editor with a hand-drawn profile
- washboard: Sine Wave block
- two bumps: Pulse Generator (smoothed with a filter) or concatenated segments in Signal Editor
- rough road: Band-Limited White Noise block (optionally followed by a Transfer Fcn / Discrete Filter block to shape the spectrum)

Data-driven (optional):

- Import a recorded profile from CSV (e.g., `readtable`) and feed it via From Workspace.
- Deliverable: `roadSuite.m` (or `roadSuite.mat`) that produces named road inputs, plus a short note explaining how each profile was made.

```
% Insert your code here (or make use of helper functions or additional .m
or .mlx files and indicate where
% they can be found). Ensure your code is well-documented.
```

Task 3: Log signals and define objective metrics

Log at minimum:

- Sprung-mass vertical acceleration
- Suspension travel (relative displacement)
- Tire deflection (relative wheel-to-road displacement) or normal-force proxy

Compute metrics per road case, such as:

- Comfort metric: RMS sprung acceleration (and/or peak)
- Packaging metric: max suspension travel
- Road-holding metric: max tire deflection (or variability)

Deliverable: a function like:

- `results = scoreSuspension(simout, roadName)`
- returning a struct/table of metrics and a single “score”

```
% Insert your code here (or make use of helper functions or additional .m
or .mlx files and indicate where
% they can be found). Ensure your code is well-documented.

function results = scoreSuspension(simout, roadName)
    % Write your function here
end
```

Task 4: Build an automated test runner

Create a script/function that:

- Runs all road cases automatically
- Logs results to a table
- Outputs a clear pass/fail for each requirement
- Generates a single summary figure (or a brief report)

Example deliverable: `runAllTests.m` → returns `summaryTable` and saves plots to `/results`

```
% Here, it is recommended to create a separate runAllTests.m file
% (or include as a local helper function) that combines the work you've
done
% so far here and additionally performs the steps outlined in the task
description.
% You can run it from here.
```

Task 5: Tune suspension parameters

Tune at least:

- Suspension spring stiffness `Ks`
- Suspension damping `Cs`

Two tuning options:

Option A — Parameter sweep (recommended for beginners)

- Sweep `Ks` and `Cs` across small ranges
- Score each design across all road cases
- Select a design that meets constraints and minimizes score

You can implement the parameter sweep in MATLAB using `Simulink.SimulationInput` objects to programmatically create simulation runs with different values of suspension stiffness (Ks) and damping (Cs) without modifying the model manually.

Implement the parameter sweep in MATLAB for each design point, configure the parameter values in a `SimulationInput` object, execute the sweep using `parsim` (see here for more information on Running Parallel Simulation) when Parallel Computing Toolbox is available, or `sim` as a serial fallback.

To reduce repeated runtime, use Fast Restart for interactive scripted simulations, and optionally Rapid Accelerator for faster batch execution of the design space.

Option B — Lightweight optimization (recommended for intermediate-level users)

- Use a simple search (pattern search / constrained minimization / custom heuristic)
- Minimize comfort subject to travel/deflection limits

Deliverable: chosen parameter values + justification using plots/tables.

```
% Insert your code here that will output the optimal values for suspension
% spring stiffness (Ks) and suspension damping (Cs) based on your
% simulation. Your output should also include plots/tables that justify the
% optimal parameter values.
```

Task 6: Validate robustness with one simple variation

Pick one low-effort robustness test:

- Payload change: increase sprung mass by +25% and rerun the suite
- Component tolerance: apply ±10% variation to `Ks` and `Cs` for a small Monte Carlo set (e.g., 20 trials)

Deliverable: a robustness summary (worst-case metrics and pass rate).

```
% Insert code here to change the parameters as suggested and rerun your
% test suite. Your code should return a robustness summary that includes
% worst-case metrics and pass rate.
```

Interpretation of Results

Summarize your findings by interpreting their physical/engineering meaning:

What are some limitations of your work?

What are practical next steps?